

AirGuard AI

Air Quality & Health Risk Prediction Platform

Product Requirements Document, Technical Requirements Document, UI/UX Design,
Backend Schema, Database Architecture and Deployment Guide

Prepared as a Final Year Project Report

Domain: Data Science, Time Series Forecasting, Public Health Analytics, Full Stack Development

Geographic Focus: Indian Cities with special emphasis on Nagpur, Maharashtra

Project Type	Final Year B.Tech Project
Target Users	Citizens, hospitals, researchers, authorities
Core Data Sources	CPCB, data.gov.in, weather APIs, census proxies
Suggested Stack	Next.js, FastAPI, PostgreSQL, Redis, Prophet/LSTM

Submitted by: Kuldeep Katre

Date: June 2026

Declaration

I hereby declare that this project report titled "AirGuard AI - Air Quality & Health Risk Prediction Platform" is an original work prepared for academic submission. The work described in this document is intended as a complete final-year project proposal and implementation blueprint, combining real public datasets, predictive analytics, and dashboard-based decision support.

Acknowledgement

I would like to acknowledge the guidance of faculty mentors, the availability of government open datasets, and the broader open-source ecosystem that makes it possible to build meaningful data-driven applications. This project is designed to address a real public-health problem and to demonstrate practical skills in software engineering, machine learning, and product design.

Abstract

AirGuard AI is an end-to-end predictive analytics platform focused on air quality in Indian cities. Unlike basic pollution dashboards that only show current readings, this system forecasts AQI spikes 3 to 5 days ahead using historical pollution records, weather variables, and time-series forecasting models. It also computes health-risk scores for vulnerable groups such as children, elderly citizens, and asthma patients. The platform is intended to support public awareness, health planning, and municipal decision-making through an interactive city-comparison dashboard, trend analysis, alerts, and downloadable reports.

Table of Contents

- 1. Introduction
- 2. Problem Statement and Objectives
- 3. Scope and Assumptions
- 4. PRD
- 5. TRD
- 6. UI/UX Design
- 7. Backend Schema
- 8. Database Architecture
- 9. Deployment Guide
- 10. Testing and Validation
- 11. Risks and Mitigation
- 12. Future Scope
- 13. Conclusion
- 14. References

1. Introduction

Urban air pollution is a persistent public-health problem in India. Many citizens consult AQI apps only after the pollution has already worsened. A forecasting system is more valuable because it enables preventive action. AirGuard AI addresses this gap by combining pollution history, weather conditions, and predictive modeling to estimate near-future AQI levels and associated health risks.

The project is intentionally designed around Indian open data and an Indian city context. This makes the work more credible than a generic academic demo, because the dataset, environmental conditions, and impact are real.

1.1 Project Motivation

Most student projects overuse toy datasets such as Iris or Titanic. Those projects are easy to reproduce and do not show strong initiative. AirGuard AI uses government open data and a public-health use case, which makes it more relevant for evaluation, internships, and portfolio use.

1.2 Proposed Outcome

- Forecast AQI for 1, 3, and 5 days ahead.
- Estimate health risk for vulnerable demographics.
- Compare multiple cities on a single dashboard.
- Show trend patterns and seasonal pollution behavior.
- Provide actionable recommendations and alerts.

2. Problem Statement and Objectives

Current AQI tools mostly answer the question "What is the air quality right now?" They do not answer the more useful question "What will air quality be in the next few days, and who will be at risk?"

The proposed system fills this gap by predicting future pollution trends and converting them into practical risk levels for different population groups.

2.1 Objectives

- Build a forecasting engine for AQI spikes using historical pollution and weather data.
- Create a health-risk score for children, elderly citizens, and asthma patients.
- Develop an interactive dashboard with city comparison and analytics.
- Design a scalable backend that can support future alerts, maps, and mobile access.
- Deploy the complete platform on cloud infrastructure.

2.2 Success Criteria

- Forecasts are available for supported cities with acceptable error metrics.
- The UI is responsive and understandable for non-technical users.
- The database stores historical observations, predictions, and user preferences cleanly.
- The system can be deployed publicly and updated with new data automatically.

3. Scope and Assumptions

The first release focuses on selected Indian cities such as Nagpur, Mumbai, Pune, Delhi, Hyderabad, and Bengaluru. The system assumes access to historical AQI and weather data from open or licensed APIs. Where direct admission data is unavailable, proxy indicators may be used for health-risk correlations.

3.1 In Scope

- Historical AQI ingestion and cleaning.
- Weather feature ingestion.
- Time-series forecasting.
- Risk scoring and recommendations.
- Dashboard, charts, and city comparison.

3.2 Out of Scope for MVP

- Medical diagnosis.
- Personalized treatment advice.
- Real-time emergency response dispatch.
- Direct hospital integration unless data access is available.

4. Product Requirements Document (PRD)

4.1 User Personas

Persona	Needs	Key Behavior
Citizen	Check AQI before travel or outdoor activity	Visits dashboard frequently
Parent	Protect children from exposure	Needs clear risk indicators
Asthma patient	Avoid polluted periods	Needs alerts and forecasts
Hospital planner	Prepare for possible admissions	Looks at spikes and trends
Researcher / authority	Study pollution patterns	Requires exportable data

4.2 Core Features

Feature	Description
AQI Forecasting	Predict AQI spikes 1, 3, and 5 days in advance.
Health Risk Engine	Convert predicted AQI into actionable risk scores by demographic.
City Comparison	Compare AQI, weather, and trend metrics across multiple cities.
Historical Analytics	Show seasonal patterns, moving averages, and anomaly periods.
Maps and Hotspots	Display city-level or zone-level pollution distribution.
Alerts and Reports	Enable notifications and downloadable summaries.

4.3 Functional Requirements

- The system shall allow a user to select a city and view current and predicted AQI values.
- The system shall display risk scores for vulnerable groups.
- The system shall show trend charts and city comparison visualizations.
- The system shall allow users to download summary reports.
- The system shall provide recommendations based on forecast and risk level.

4.4 Non-Functional Requirements

Attribute	Target
Performance	Typical dashboard response under 2 seconds
Availability	99.9 percent target for production use
Security	JWT auth, HTTPS, input validation, rate limiting
Scalability	Ability to add more cities and more users
Maintainability	Clear API separation and modular ML pipeline

4.5 Product Workflow

User opens the platform, selects a city, views the current AQI and forecast, checks the risk score, compares the city with others if needed, and downloads a report or sets an alert.

5. Technical Requirements Document (TRD)

5.1 Technology Stack

Layer	Recommended Tech	Reason
Frontend	Next.js, TypeScript, Tailwind CSS, Recharts	Fast, responsive, SEO-friendly
Backend	FastAPI	Python-native ML integration
ML	Pandas, NumPy, Prophet or LSTM, scikit-learn, MLflow	Forecasting and evaluation
Database	PostgreSQL	Reliable relational storage
Cache	Redis	Prediction caching and rate limits
Storage	AWS S3 or compatible storage	Datasets and model artifacts
Deployment	Vercel, Render, Docker, GitHub Actions	Simple cloud rollout

5.2 Data Sources

- CPCB or data.gov.in AQI and pollutant data.
- Weather API for temperature, humidity, wind speed, and rainfall.
- City and demographic data from census or government references.
- Optional health proxy datasets for trend correlation.

5.3 ML Pipeline

Step	Details
Data ingestion	Pull historical AQI and weather series into staging tables
Cleaning	Handle missing values, inconsistent dates, and outliers
Feature engineering	Create lag features, rolling averages, seasonal indicators
Modeling	Train Prophet baseline and LSTM advanced model
Evaluation	Use MAE, RMSE, MAPE, and backtesting
Serving	Expose prediction endpoints through FastAPI

5.4 API Requirements

Method / Endpoint	Purpose	Response
GET /api/v1/aqi/current?city=	Current AQI	Current air quality status
GET /api/v1/forecast/{city}	Forecast output	1/3/5 day predicted AQI
GET /api/v1/risk/{city}	Health risk	Risk by demographic
POST /api/v1/compare	City comparison	Multi-city analytics
GET /api/v1/history	Historical data	Trend series and chart data

5.5 Quality Attributes

- All APIs must validate inputs and reject malformed requests.
- Model versions must be tracked so forecasts are reproducible.
- The backend must be designed so that new cities can be added without code changes.
- Logs and error handling must support debugging and operational monitoring.

6. UI/UX Design

The UI must be clean, data-first, and readable on both desktop and mobile. Since the project is aimed at public awareness, the interface should avoid clutter and explain risk levels in plain language.

6.1 Primary Screens

- Home dashboard with city search and live AQI cards.
- Forecast page with line chart and confidence band.
- Comparison page for multiple cities.
- Historical trends page with filters and date range.
- Settings / alerts page for notification preferences.

6.2 Visual Language

- Green to red AQI status colors.
- Card-based layout with strong hierarchy.
- Charts placed above dense tables.
- Short recommendations beneath forecasts.
- Simple language suitable for non-technical users.

6.3 Wireframe Description

The dashboard starts with a city selector and current AQI card. Below that, there is a forecast chart, a risk summary by demographic, trend analytics, and a side panel with recommendations. The comparison page uses a table and charts so that users can evaluate multiple cities quickly.

7. Backend Schema

The backend should be organized into modular services: authentication, city management, AQI ingestion, forecasting, risk scoring, report generation, and notification delivery.

7.1 Service Modules

- Auth Service - login, token issuance, role-based access.
- City Service - city registry, location metadata, comparison support.
- AQI Service - current readings, historical records, pollutant values.
- Forecast Service - prediction generation and caching.
- Risk Service - demographic risk score calculations.
- Report Service - PDF/CSV exports.
- Notification Service - email or push alerts.

7.2 API Layer Structure

Recommended route groups: /auth, /cities, /aqi, /forecast, /risk, /compare, /reports, /alerts. Each group should have separate request validation and response models.

8. Database Architecture

A relational database is the right default choice because the system has structured city, forecast, and user data. PostgreSQL is preferred for its reliability and support for analytics-friendly querying.

8.1 Core Tables

Table	Purpose	Key Columns
users	Login and alert preferences	id, name, email, password_hash, role
cities	Supported city metadata	id, city_name, state, lat, lon
aqi_records	Historical pollutant data	city_id, date, pm25, pm10, no2, so2, co, o3, aqi
weather_records	Weather features	city_id, date, temp, humidity, rainfall, wind_speed
predictions	Forecast outputs	city_id, forecast_date, predicted_aqi, confidence
risk_scores	Health-risk summaries	prediction_id, children_score, elderly_score, asthma_score
alerts	User subscriptions	user_id, city_id, threshold, channel

8.2 Entity Relationships

A city has many AQI records, many weather records, and many predictions. A prediction has one or more risk scores. Users may subscribe to alerts for one or more cities. Normalization should be maintained so that city metadata is not duplicated inside time-series records.

8.3 Data Retention Policy

- Raw observations should be retained for auditability.
- Aggregated analytics can be stored for quick dashboard queries.
- Old model outputs can be archived but not deleted immediately.
- User-generated alert preferences must remain editable and secure.

9. Deployment Guide

The recommended deployment strategy uses containerization so the platform can be reproduced locally and in cloud environments. Frontend and backend should be deployed separately for easier scaling.

9.1 Local Development

- Use Docker Compose for backend, database, cache, and optionally a mock data service.
- Run the frontend independently with the production backend URL configured through environment variables.
- Keep secrets in .env files and never commit them to source control.

9.2 Production Deployment

Component	Recommended Platform	Notes
Frontend	Vercel	Simple deploys and fast CDN delivery
Backend API	Render or AWS EC2	Dockerized FastAPI service
Database	Neon, Supabase, or AWS RDS	Managed PostgreSQL
Cache	Upstash Redis	Managed cache and rate limiting
Storage	AWS S3	Artifacts, reports, and model files
CI/CD	GitHub Actions	Build, test, deploy automation

9.3 Deployment Steps

1. Containerize backend and test locally.
2. Configure PostgreSQL and Redis connections.
3. Push code to GitHub with environment variables documented.
4. Deploy backend and verify health endpoint.

5. Deploy frontend with production API base URL.
6. Run smoke tests for dashboard, forecast, and comparison pages.

9.4 Monitoring

- Log API errors and model failures.
- Use uptime monitoring for backend health.
- Track model drift and retraining cadence.
- Monitor database growth and cache hit rates.

10. Testing and Validation

Testing should cover correctness, usability, and model quality. A project of this type is not complete if only the UI works but the forecast is unreliable.

10.1 Test Types

- Unit testing for data cleaning and risk calculations.
- API testing for endpoint validation and response formats.
- Integration testing for database and model serving.
- UI testing for responsive behavior and chart rendering.
- Model evaluation using time-based backtesting.

10.2 Example Metrics

Metric	Use	Target
MAE	AQI forecast error	Lower is better
RMSE	Penalize large misses	Lower is better
MAPE	Percentage error	Below acceptable threshold
Latency	Dashboard responsiveness	< 2 seconds
Uptime	Availability	99.9 percent target

11. Risks and Mitigation

Risk	Impact	Mitigation
Missing data	Model gaps	Use imputation and backup sources
API limits	Ingestion failures	Cache data and schedule pulls
Model drift	Reduced accuracy	Retrain periodically
No hospital data	Weak health linkage	Use proxy indicators and clearly label them
UI complexity	User confusion	Use simple cards and clear risk labels

12. Future Scope

- Add zonal pollution maps using GIS layers.
- Integrate additional cities and longer time horizons.
- Add SMS / WhatsApp notifications.
- Train separate models for seasonal and city-specific behavior.
- Create a mobile application for public alerts.
- Add health-center workload forecasting if hospital data becomes available.

13. Conclusion

AirGuard AI is a strong final-year project because it solves a realistic problem using real Indian open data, forecasting, public-health reasoning, and full-stack system design. It is more credible than a standard CRUD application and more useful than a dataset-only ML notebook. With proper implementation, deployment, and documentation, it can serve as a portfolio flagship project.

14. References

- Central Pollution Control Board (CPCB) open data resources.
- Government of India data portal and public environmental datasets.
- Weather API documentation for climate features.
- Prophet / time-series forecasting documentation.
- FastAPI, PostgreSQL, Redis, Docker, GitHub Actions documentation.

Appendix A - Suggested Deliverables

- Source code repository with README.
- Dataset ingestion scripts.
- Model training notebook or pipeline.
- Backend API documentation.
- UI screenshots and dashboard video demo.
- Final report PDF and project presentation slides.